Caso: Classificação com Perceptron na scikit-learn

Aula 06: Introdução ao Processamento de Linguagem Natural (PLN)

Prof. Eduardo Corrêa Gonçalves

19/03/2026

Sumário

Introdução

Analisador de Sentimentos com Perceptron: passo a passo

- Base de dados
- Treinamento do classificador
- Estudo das *features* geradas
- Classificação com o modelo treinado

Caso: Análise de Sentimentos

- **Apresentação do Exemplo**
  - Nesta aula, apresentaremos um exemplo de classificação de sentimentos na sklearn, que foi projetado da maneira a seguir:
    - **Estruturação do texto**: geração 2 *features* (as mesmas mostradas no exemplo da aula teórica sobre o Perceptron):
      - Total de palavras positivas
      - Total de palavras negativas
    - **Algoritmo de Classificação**: Perceptron (aula 04)
    - **Base de dados (BD)**: "sentiment140", uma BD real utilizada em muitos artigos.
  - **Obs.**: Embora o Perceptron seja um algoritmo limitado, ele serve como técnica inicial para o estudo das Redes Neurais.

Link para notebook com os exemplos da aula:

- O código apresentado nos slides a seguir está disponível no seguinte notebook:

- https://colab.research.google.com/drive/1Elu5UsJc-fL1wSFXFEjCYDI3nlFJJyPD?usp=sharing

Base de Dados

- **<u>Base:</u> Sentiment140**
  - Usaremos a base CSV chamada **Sentiment140**
  - Foi introduzida no paper "Twitter Sentiment Classification using Distant Supervision" (Go et al., 2009)
  - Possui 2 colunas:
    - 1ª - sentimento (*sentiment*)
    - 2ª - texto (review)
  - São 359 textos, sendo 182 positivos e 187 negativos
    - Base bem balanceada

	sentiment	review
0	positive	@stellargirl I loooooooovvvvvveee my Kindle2. ...
1	positive	Reading my kindle2... Love it... Lee childs i...
2	positive	Ok, first assesment of the #kindle2 ...it fuck...
3	positive	@kenburbary You'll love your Kindle2. I've had...
4	positive	@mikefish Fair enough. But i have the Kindle2...
...	...	...
354	positive	After using LaTeX a lot, any other typeset mat...
355	negative	On that note, I hate Word. I hate Pages. I hat...
356	positive	Ahhh... back in a *real* text editing environm...
357	negative	Trouble in Iran, I see. Hmm. Iran. Iran so far...
358	negative	Reading the tweets coming out of Iran... The w...

Treinamento do Classificador – passo a passo (1/14)

- **Novamente, utilizaremos <u>4 passos</u> para <u>treinar</u> o classificador na scikit-learn**

  1. Importação da base de dados de treinamento.
  2. Separação das partes X (*features*) e Y (classe).
  3. Pré-processamento da parte X (+ estudo das *features* geradas)
  4. Treinamento do modelo.

- O pré-processamento consistirá em transformar os textos em 2 variáveis (total de palavras positivas e total de palavras negativas).

- **Conforme comentado na aula 04, não** é um **esquema fixo**! Dependendo do problema, você pode alterar esse fluxo.

Treinamento do Classificador – passo a passo (2/14)

- **<u>PASSO 1</u>: importação da base de dados (BD) rotulada**
  - O arquivo CSV será importado para um **DataFrame** pandas (código abaixo).
  - Basta especificar diretamente o endereço da base no repositório GitHub.

```
import pandas as pd

# ------------------------------------------------------------
# Importa os dados de treinamento para um DataFrame
# ------------------------------------------------------------
arq = https://raw.githubusercontent.com/edubd/bd/refs/heads/main/sentiment140.csv'

df = pd.read_csv(arq)

df # para visualizar o conteúdo
```

Treinamento do Classificador – passo a passo (3/14)

- **<u>Resultado</u> ao final do <u>PASSO 1</u>**
  - BD de treino terá sido importado p/ DataFrame

	sentiment	review
0	positive	@stellargirl I loooooooovvvvvveee my Kindle2. ...
1	positive	Reading my kindle2... Love it... Lee childs i...
2	positive	Ok, first assesment of the #kindle2 ...it fuck...
3	positive	@kenburbary You'll love your Kindle2. I've had...
4	positive	@mikefish Fair enough. But i have the Kindle2...
...	...	...
354	positive	After using LaTeX a lot, any other typeset mat...
355	negative	On that note, I hate Word. I hate Pages. I hat...
356	positive	Ahhh... back in a *real* text editing environm...
357	negative	Trouble in Iran, I see. Hmm. Iran. Iran so far...
358	negative	Reading the tweets coming out of Iran... The w...

359 rows × 2 columns

Treinamento do Classificador – passo a passo (4/14)

- **<u>PASSO 2</u>: separação das partes X (*features*) e Y (classe)**
  - Como sabemos, para treinar o modelo a sklearn exige que seja indicado quem é o conjunto de *features* X e quem é a classe Y. Então, p/ facilitar, fazemos a separação.
    - Veja que chamei X de "**X_raw**"... É porque ele contém o que chamamos de "textos brutos" ou "textos crus"
      - Precisará ser pré-processado antes do treinamento do modelo de classificação

```
# Separação em: X_raw (X "cru", ainda não pré-processado)
#               Y     (classe)
X_raw = df.loc[:, ['review']] # DataFrame com a coluna "review"
Y = df.iloc[:, 0]  # Series com a coluna "sentiment"
```

Treinamento do Classificador – passo a passo (5/14)

- **Resultado ao final do PASSO 2**
  - X_raw (textos originais dos comentários) e Y (classe de cada texto) terão sido separados.

```
print('- X (original):'); print(X_raw)
print('- Y:'); print(Y)
```

```
- X raw:
                                                  review
0  @stellargirl I loooooooovvvvvveee my Kindle2. ...
1  Reading my kindle2...  Love it... Lee childs i...
2  Ok, first assesment of the #kindle2 ...it fuck...
3  @kenburbary You'll love your Kindle2. I've had...
4  @mikefish  Fair enough. But i have the Kindle2...
(359, 1)
```

```
- Y:

0      positive
1      positive
2      positive
3      positive
4      positive
          ...
```

Treinamento do Classificador – passo a passo (6/14)

- **PASSO 3: Pré-processamento da parte X**
  - Quase sempre, passo mais trabalhoso em projetos reais de PLN e IA.
  - Cada texto será representado por **2 *features*** [x_1, x_2]
    - x_1 = total de palavras positivas no texto
    - x_2 = total de palavras negativas no texto
  - Essas *features* (junto com outras) são "clássicas" na área de análise de sentimentos e constam no exemplo de Jurafsky e Martin (2026).
    - Elas são **interpretáveis**, por são uma boa opção para combinar não somente com o Perceptron, mas com algoritmos que geram modelos interpretáveis, como árvore de decisão e regressão logística.

Treinamento do Classificador – passo a passo (7/14)

- **PASSO 3: Pré-processamento da parte X**
  - (3.1) o texto original é **convertido para minúsculo** e **transformado em uma lista de palavras e símbolos** (tokens), como no exemplo abaixo:
    - "Eu vou. Por que não? Por que não?" → ["eu", "vou", ".", "por", "que", "não", "?", "por", "que", "não", "?"]
    - Esse processo é conhecido como **tokenização**. No exemplo, fizemos com a biblioteca NLTK que possui uma função pronta para tokenizar (*detalhes em uma aula futura*)

```
# importa a biblioteca nltk

import nltk
nltk.download('punkt_tab')

# (1) converte para minúsculo e tokeniza os textos brutos
(x_raw)
def tokeniza(texto):
  return nltk.word_tokenize(texto.lower())

X_raw['tokens'] = X_raw['review'].apply(tokeniza)
```

Treinamento do Classificador – passo a passo (8/14)

- **Resultado da tokenização**
  - Uma nova coluna chamada "tokens" com a lista de tokens terá sido adicionada ao DataFrame "X_raw".

	review	tokens
0	@stellargirl I loooooooovvvvvveee my Kindle2. ...	[@, stellargirl, i, loooooooovvvvvveee, my, ki...
1	Reading my kindle2... Love it... Lee childs i...	[reading, my, kindle2, ..., love, it, ..., lee...
2	Ok, first assesment of the #kindle2 ...it fuck...	[ok, ,, first, assesment, of, the, #, kindle2,...
3	@kenburbary You'll love your Kindle2. I've had...	[@, kenburbary, you, 'll, love, your, kindle2,...
4	@mikefish Fair enough. But i have the Kindle2...	[@, mikefish, fair, enough, ., but, i, have, t...
...	...	...
354	After using LaTeX a lot, any other typeset mat...	[after, using, latex, a, lot, ,, any, other, t...
355	On that note, I hate Word. I hate Pages. I hat...	[on, that, note, ,, i, hate, word, ., i, hate,...
356	Ahhh... back in a *real* text editing environm...	[ahhh, ..., back, in, a, *, real, *, text, edi...
357	Trouble in Iran, I see. Hmm. Iran. Iran so far...	[trouble, in, iran, ,, i, see, ., hmm, ., iran...
358	Reading the tweets coming out of Iran... The w...	[reading, the, tweets, coming, out, of, iran, ...

359 rows × 2 columns

Treinamento do Classificador – passo a passo (9/14)

- **PASSO 3: Pré-processamento da parte X**
  - (3.2) Cada texto será transformado em 2 meta-variáveis $[x_1, x_2]$
  - Para criá-las, precisamos passar para o programa uma lista contendo palavras positivas, uma lista com palavras negativas.
  - É bastante fácil encontrar na Internet... Vou usar as listas de Hu & Liu (2004) → http://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html

1	a+
2	abound
3	abounds
4	abundance
5	abundant
6	accessable
7	accessible
8	acclaim
9	acclaimed
10	acclamation
11	accolade
12	accolades
13	accommodative
14	accomodative
15	accomplish

Trecho do léxico de palavras **positivas**

1	2-faced
2	2-faces
3	abnormal
4	abolish
5	abominable
6	abominably
7	abominate
8	abomination
9	abort
10	aborted
11	aborts
12	abrade
13	abrasive
14	abrupt
15	abruptly

Trecho do léxico de palavras **negativas**

Treinamento do Classificador – passo a passo (10/14)

- **PASSO 3: Pré-processamento da parte X**
  - (3.2) o código a seguir importa os léxicos de palavras positivas e negativas para uma lista.

```
# lê o dicionário de palavras positivas e o converte p/ 1 lista
pos =
pd.read_csv("https://raw.githubusercontent.com/edubd/bd/refs/he
ads/main/pos.txt",header=None, names = ['palavra’])

pos = pos['palavra'].to_list()

# lê o dicionário de palavras negativas e o converte p/ 1 lista
neg =
pd.read_csv("https://raw.githubusercontent.com/edubd/bd/refs/he
ads/main/neg.txt",header=None, names = ['palavra’])

neg = neg['palavra'].to_list()
```

Treinamento do Classificador – passo a passo (11/14)

- **PASSO 3: Pré-processamento da parte X**
  - (3.3) Agora cria-se uma função para cada meta-variável
    - As colunas x1 e x2 serão criadas aplicando as suas respectivas funções.

```
# (3.3) define as funções para gerar as features numéricas

def f_gera_x1(tokens):
  # quantidade de palavras positivas
  return sum([1 if token in pos else 0 for token in tokens])

def f_gera_x2(tokens):
  # quantidade de palavras negativas
  return sum([1 if token in neg else 0 for token in tokens])
```

Treinamento do Classificador – passo a passo (12/14)

- **PASSO 3: Pré-processamento da parte X**
  - (3.4) Aplica-se as funções !!!

```
# (3.4) aplica as funções
X = X_raw.copy()
X['x1'] = X['tokens'].apply(f_gera_x1)
X['x2'] = X['tokens'].apply(f_gera_x2)
```

Resultado ao final do Passo 3: X pré-processado

	review	x1	x2
0	@stellargirl I loooooooovvvvvveee my Kindle2. ...	3	0
1	Reading my kindle2... Love it... Lee childs i...	2	0
2	Ok, first assesment of the #kindle2 ...it fuck...	0	1
3	@kenburbary You'll love your Kindle2. I've had...	1	1
4	@mikefish Fair enough. But i have the Kindle2...	3	0

Estudando as *features* geradas (1/4)

- **<u>EXTRA</u>:** Vamos estudar as variáveis x_1 e x_2.
  - I. **frequência cruzada**
    - Maior parte dos reviews tem no máximo 2 palavras positivas ou negativas.

```
print('Freq. cruzada (x1 - colunas, x2 - linhas):')
cross_freq = pd.crosstab(index=X['x2'], columns=X['x1'])
pd.DataFrame(cross_freq)
```

Freq. cruzada (x1 - colunas, x2 - linhas):

x1	0	1	2	3	4	5
x2						
0	64	82	37	10	2	1
1	55	43	11	3	1	0
2	29	6	0	1	0	0
3	5	5	0	1	0	0
4	2	0	0	0	0	0
5	1	0	0	0	0	0

Estudando as *features* geradas (2/4)

- **EXTRA**: Vamos estudar as variáveis x_1 e x_2.
  - II. Verificando a **freq. das classes** para diferentes valores de x1 e x2
    - Quando o número de palavras negativas e positivas é igual, não há uma classe realmente predominante.

```
print('docs sem palavras positivas ou negativas')
filtro1 = ((X['x1'] == 0) & (X['x2'] == 0))
Y_f1 = Y[filtro1]
print(Y_f1.value_counts())
```

```
docs sem palavras positivas ou negativas
sentiment
positive    33
negative    31
Name: count, dtype: int64
```

```
print('docs com o mesmo número de positivas e negativas')
filtro4 = (X['x1'] == X['x2'])
Y_f4 = Y[filtro4]
print(Y_f4.value_counts())
```

```
docs com o mesmo número de positivas e negativas
sentiment
negative    55
positive    53
Name: count, dtype: int64
```

Estudando as *features* geradas (3/4)

- **EXTRA**: Vamos estudar as variáveis x_1 e x_2
  - II. Verificando a **freq. das classes** para diferentes valores de x1 e x2
    - Quando há 0 palavras negativas (positivas) e mais de 1 positiva (negativa), há uma classe claramente predominante

```
print('docs c/ ao menos 1 positiva e 0 negativas')
filtro3 = ((X['x1’] >= 1) & (X['x2’] == 0))
Y_f3 = Y[filtro3]
print(Y_f3.value_counts())
```

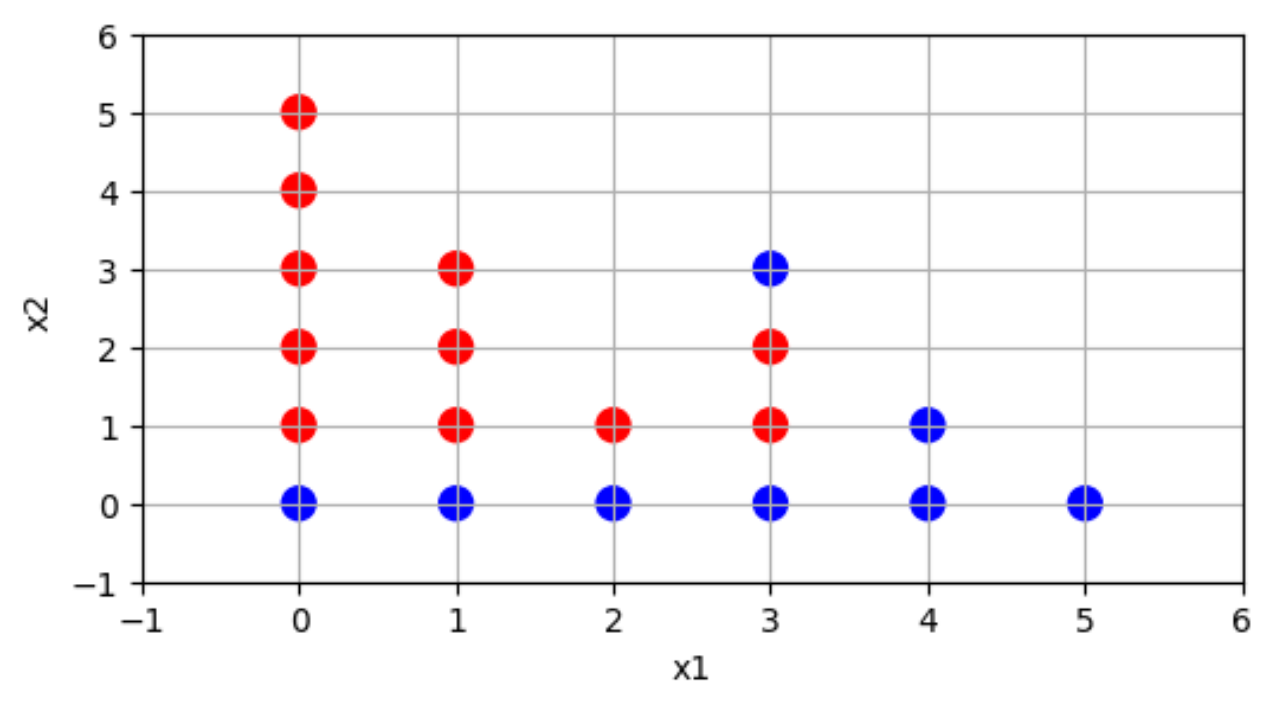
```
docs c/ ao menos 1 positiva e 0 negativas
sentiment
positive    116
negative     16
Name: count, dtype: int64
```

```
print('docs c/ 0 positivas e ao menos 1 negativa')
filtro3 = ((X['x1'] == 0) & (X['x2'] >= 1))
Y_f3 = Y[filtro3]
print(Y_f3.value_counts())
```

```
docs c/ 0 positivas e ao menos uma negativa
sentiment
negative    86
positive     6
Name: count, dtype: int64
```

Estudando as *features* geradas (4/4)

- **<u>EXTRA</u>:** Plotando a classe predominante p/ cada par $\{x_1,x_2\}$
  - III. : Plotando a classe predominante p/ cada par $\{x_1,x_2\}$
    - vermelho: sentimento negativo é predominante.
    - azul: sentimento positivo é predominante.
      - *Ver código no notebook*
    - Fronteira que separa os docs negativos e positivos não é linear



IBGE
Escola Nacional de Ciências Estatísticas ENCE

Treinamento do Classificador – passo a passo (13/14)

- **PASSO 4: Treinamento do classificador**
  - (1). **importar o pacote** com o algoritmo que você quer usar, nesse caso o Perceptron (pacote `linear_model.Perceptron` da scikit)
  - (2). **Instanciar o classificador**
    - O parâmetro random_state = 0 é usado para que toda vez que o código seja rodado, os mesmos pesos iniciais sejam gerados.
  - (3). **Usar o método** (função) **fit**, para fazer o treinamento do classificador, indicando o X e o Y. Nesse caso:
    - Para X, passamos apenas as colunas x1 e x2
    - Y é o vetor com as classes, que já separamos lá no Passo 2.

```
#1. importa o pacote com a classe Perceptron
from sklearn.linear_model import Perceptron

#2. instancia um objeto da classe Perceptron
#   random_state = 0 é usado para que sempre os mesmos
#   pesos iniciais serão gerados
modelo = Perceptron(random_state=0)

#3. utiliza o método "fit" para treinar o modelo
#   passamos apenas as colunas x1 e x2 de X
modelo.fit(X[['x1', 'x2']], Y)
```

Treinamento do Classificador – passo a passo (14/14)

- **Resultado ao final do PASSO 4**
  - **Classificador Perceptron está construído!!!**
  - Agora podemos utilizá-lo.
  - Abaixo um código onde imprimimos os parâmetros do modelo, ou seja w_1, w_2 e b:

```
# printa os parâmetros
print('parâmetros do modelo:')
print('w1 e w2: ', modelo.coef_)
print('b: ', modelo.intercept_)
```

```
Modelo treinado com sucesso!
parâmetros do modelo:
w1 e w2:  [[ 3. -3.]]
b:  [2.]
```

- Ou seja, o **modelo** é:
  - $y = 3x_1 - 3x_2 + 2$
  - **Basicamente**: se tiver mais positivas classifica como positivo e mais negativas como negativo
  - Se empatar, considera positivo (pesos iguais, $b = 2$)
  - *Não é um modelo muito útil...*

Classificação de novos textos – passo a passo (1/4)

- **Os <u>3 passos</u> para <u>utilizar</u> um classificador treinado na scikit-learn**
    1. Definição ou importação dos novos documentos (textos) – *ou seja os documentos a serem classificados pelo modelo*
    2. Pré-processamento dos novos documentos
    3. Utilização do modelo treinado para classifica-los

- Mais uma vez, **não** é um **esquema fixo**!
- Dependendo do problema ou da sua preferência, você pode alterar esse fluxo.

Classificação de novos textos – passo a passo (2/4)

- **<u>Passo 1:</u> Definição ou importação dos novos documentos**
  - Em nosso exemplo, defini manualmente um documento novo em um DataFrame: *"I really disliked this horrible phone"*
  - Mas também é possível importar um dataset com vários documentos novos (na sklearn, posso aplicar o modelo para classificar vários textos de uma vez)

```
novoDoc = pd.DataFrame({"review": ["I really disliked this
horrible phone"]})
print('- Novo documento:'); print(novoDoc)
print()
```

```
- Novo documento:
                                    review
0  I really disliked this horrible phone
```

Classificação de novos textos – passo a passo (3/4)

- **<u>Passo 2:</u> Pré-processamento dos novos documentos**
  - Agora, temos que gerar x1 e x2 para o documento.
    - Primeiro chamamos a função de tokenização e depois as funções que geram x1 e x2

```
novoDoc['tokens'] = novoDoc['review'].apply(tokeniza)
novoDoc['x1'] = novoDoc['tokens'].apply(f_gera_x1)
novoDoc['x2'] = novoDoc['tokens'].apply(f_gera_x2)

# exibe as colunas criadas
print('- Novo documento pré-processados, com as features x1 e x2')
print(novoDoc[['review', 'x1', 'x2']])
```

```
- Novo documento pré-processados, com as features x1 e x2
                                 review  x1  x2
0  I really disliked this horrible phone   0   2
```

Classificação de novos textos – passo a passo (4/4)

- **Passo 3: Classificação dos novos documentos**
  - Consiste em **aplicar o modelo** treinado (em nosso caso um Perceptron) para **classificar o novo documento**.
  - Isso é feito com o método predict do modelo.
    - Ele vai retornar uma das classes possíveis: “negative” ou “positive”
  - **IMPORTANTE**: podemos fazer o predict de vários documentos novos de uma vez... Nesse caso, seria retornada uma lista contendo a classe predita para cada um deles.

```
classe_predita = modelo.predict(novoDoc[['x1', 'x2']])
print('- A classe do novo documento é: ', classe_predita)
```

```
- A classe do novo documento é:  ['negative']
```

Referências

- JURAFSKY, D.; MARTIN, J. H. Logistic Regression. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. *draft* [*s.l.*] : [*s.n.*], 2026. cap 4, p. 6-8. Disponível em: https://web.stanford.edu/~jurafsky/slp3/. Acesso em: 10 mar. 2026.

- DE OLIVEIRA, M. A.; MELO JR., F. A P. Perceptron. *In*: DE OLIVEIRA, M. A.; MELO JR., F. A P. **Backpropagation e Redes Neurais – Volume 1:** MLP, RPROP, LIMBP, DELTA-BAR-DELTA, MATLAB e PYTHON. Rio de Janeiro: Ciência Moderna, 2024. cap 3, p. 92-142.

- scikit-learn – Linear Models - Perceptron. Disponível em: https://scikit-learn.org/stable/modules/linear_model.html#perceptron. Acesso em: 13 mar. 2026

- scikit-learn - Perceptron. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html. Acesso em: 13 mar. 2026